

Projet patron de conception

Jérémy BROCHARD, Gabriel DURAND, Youssouf DIARE

Université de Caen

10 avril 2026

Contents

1	Présentation du projet	2
1.1	Contexte :	2
1.2	Présentation du plan du rapport :	2
1.3	Architecture du projet :	2
2	Modèle	2
2.1	Implémentation des formes	2
2.2	Pattern Observer	2
2.3	Plateau de jeu	2
2.4	Collision / Pattern Visiteur	2
2.5	Méthode ia	3
2.6	Pattern Strategy	3
2.6.1	StratGen1	3
2.6.2	StratGen2	4
3	Vue	5
3.1	GameMenu :	5
3.2	GameCharacter :	6
3.3	GameSettings :	7
3.4	GamePanel :	7
4	Contrôleur	8
4.1	Pattern Command	8
4.2	Pattern State	8
5	Engine	8
6	Répartition du travail	9

1 Présentation du projet

1.1 Contexte :

Ce projet, réalisé dans le cadre du module Patrons de conception et interactions homme-machine, consiste à développer un jeu 2D dans lequel le joueur doit placer des formes géométriques afin de maximiser la surface occupée sans intersection avec des obstacles.

1.2 Présentation du plan du rapport :

Dans une première partie, nous décrirons le modèle de notre jeu. Nous aborderons ensuite la partie vue, puis nous terminerons par la partie contrôleur.

1.3 Architecture du projet :

Le projet s'articule en cinq packages, eux-mêmes composés de sous-packages, répartis entre le modèle, le contrôleur et la vue.

2 Modèle

2.1 Implémentation des formes

Notre sous-package **form** est constitué d'une interface **Shape** et de deux classes **RectangleShape** et **CircleShape**, qui permettent de représenter les formes du jeu. Ces classes implémentent les opérations de déplacement, de redimensionnement et de calcul de surface (utile pour le calcul du score). Elles définissent également la méthode **contains**. Elle est utilisée lors d'un clic sur la forme afin de déterminer si le point sélectionné appartient à celle-ci.

2.2 Pattern Observer

Nous avons implémenté le pattern **Observer** afin de rendre notre plateau, nos points et nos formes écoutables. L'interface **ModelEcoutable** permet d'ajouter des écouteurs, tandis que **EcouteurModel** définit la méthode appelée lors d'un changement d'état. La classe abstraite **AbstractModeleEcoutable** centralise cette logique en maintenant une liste d'observateurs et en les notifiant automatiquement via la méthode **fireChange()**. Ainsi, dès qu'une modification (déplacement, ajout ou suppression de formes, redimensionnement) a lieu, notre plateau en est informé et ce dernier prévient notre vue afin que celle-ci puisse s'actualiser.

2.3 Plateau de jeu

La classe Plateau représente le cœur du modèle du projet Shape Wars, car elle possède tous les attributs liée au déroulement de la partie et à l'état global du plateau de jeu. En héritant de **AbstractModeleEcoutable**, elle s'intègre au pattern Observer et peut ainsi notifier la vue quand il y a un changement. Elle gère les dimensions du plateau à partir de la taille de l'écran, conserve la liste des joueurs, identifie le joueur courant, stocke les obstacles fixes ainsi que les formes placées, suit le nombre de pièces restantes grâce au compteur `piece`, et utilise une stratégie de génération d'obstacles instancier à l'initialisation de la classe.

Son rôle principal se résume à plusieurs fonction: la gestion des joueurs, la génération dynamique des obstacles grâce à `genererObs()`, qui délègue ce travail à la stratégie choisie, la vérification des collisions via la méthode `collision()`, ainsi que la gestion des formes et du score à travers `ajouterFormePlacee()`, `supprimerFormePlacee()` et `score_game()`.

2.4 Collision / Pattern Visiteur

Pour gérer la logique des collisions entre les formes dans le jeu, nous utilisons le pattern Visiteur. Celui-ci permet de résoudre le problème du calcul des collisions entre différents types de formes, en adaptant le traitement selon les types comparés. Les formes étant différentes, chaque combinaison (cercle/cercle,

rectangle/rectangle, cercle/rectangle) nécessite un algorithme de collision spécifique afin de déterminer s'il y a une intersection entre elles.

Sans le pattern Visiteur, il faudrait multiplier les `instanceof` dans chaque classe de forme, ce qui complexifie grandement la logique. La solution est la suivante : l'interface `IntersectionVisiteur` déclare deux méthodes **`visit(CircleShape)`** et **`visit(RectangleShape)`**. La classe `CollisionVisiteur` implémente cette interface : elle reçoit dans son constructeur la seconde forme à comparer (`other`), puis choisit l'algorithme compatible selon la paire que l'on compare via **`visit`**.

Chaque forme (`CircleShape`, `RectangleShape`) implémente une méthode **`accept(IntersectionVisiteur)`** qui appelle **`visitor.visit(this)`**, ce qui permet au visiteur de connaître le type concret de la première forme. La classe utilitaire `CollisionUtil` gère le tout : **`CollisionUtil.intersects(a, b)`** crée un **`CollisionVisiteur(b)`**, appelle **`a.accept(visitor)`**, puis retourne **`visitor.getResult()`**. Ainsi, ajouter un nouveau type de forme ne nécessite que d'ajouter une méthode **`visit`** dans le visiteur, sans modifier les classes existantes.

2.5 Méthode ia

La structure de l'intelligence artificielle repose sur le patron Stratégie (Strategy). L'IA doit pouvoir adopter différents comportements de placement de formes (aléatoire, barycentrique, heuristique). L'interface `PlacementStrategy` déclare une unique méthode **`placeShape(Plateau)`** qui retourne la Shape à placer. Une implémentation existe : `BarycentrePlacementStrategy`, qui calcule le barycentre de toutes les formes déjà placées sur le plateau (ou le centre du plateau si aucune forme n'est présente) et tente de placer la nouvelle forme au plus près de ce barycentre en augmentant progressivement la taille de recherche à chaque tentative infructueuse (jusqu'à 50 essais), ou un placement aléatoire en dernier recours.

La classe `ia` (qui étend `Entity`) possède un attribut `placementStrategy` de type `PlacementStrategy`. Sa méthode **`jouer()`** appelle simplement **`placementStrategy.placeShape(plateau)`** quatre fois de suite (4 pièces par tour), puis ajoute chaque forme au plateau via **`plateau.ajouterFormePlacee()`**. La stratégie peut être changée grâce à **`setPlacementStrategy()`**. Dans `GameSession`, le tour de l'IA est déclenché automatiquement via **`playIfAI()`** avec **`SwingUtilities.invokeLater()`** pour s'assurer que l'interface graphique a rendu les formes avant le prochain placement.

2.6 Pattern Strategy

Strategie est un pattern comportemental. Dans le cadre de notre projet nous l'utilisons dans le but de générer différent plateau de jeu à chaque niveau.

2.6.1 StratGen1

Cette stratégie est implémenter dans la classe `StratGen1` et elle consiste à lire des fichiers de configuration et crée des obstacles en conséquence. Cela se fait grâce à la classe `ConfigFileSelector` qui repose sur le pattern Singleton ce qui permet de ne créer qu'une seule instance de `ConfigFileSelector` dans tout le programme.

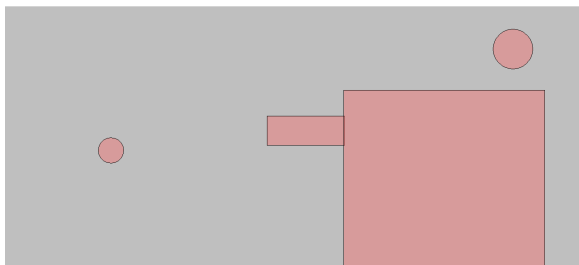


Figure 1: plateau niveau 1

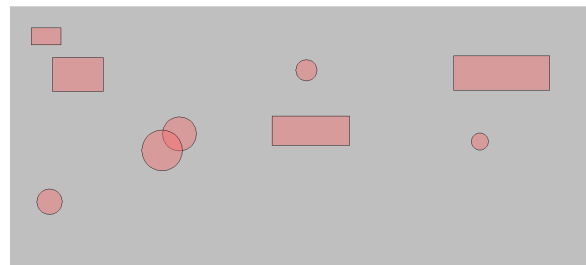


Figure 2: plateau niveau 2

2.6.2 StratGen2

Cette stratégie permet de générer des obstacles sur un plateau de jeu de manière dynamique aléatoirement. Son paramètre nbRedShapes est utilisé dans GameSetting dans le but de pouvoir créer un nombre de formes déterminé.

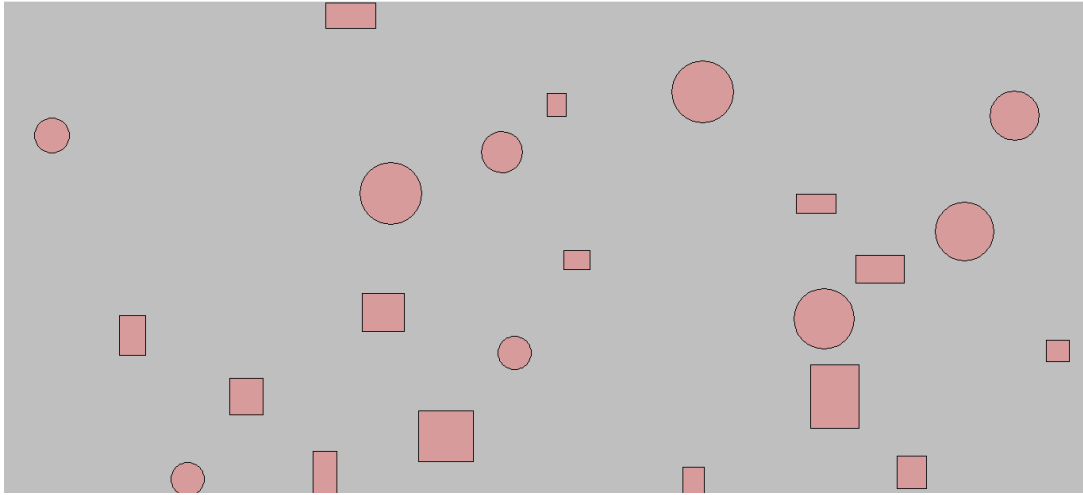


Figure 3: Génération de 25 obstacles aléatoirement

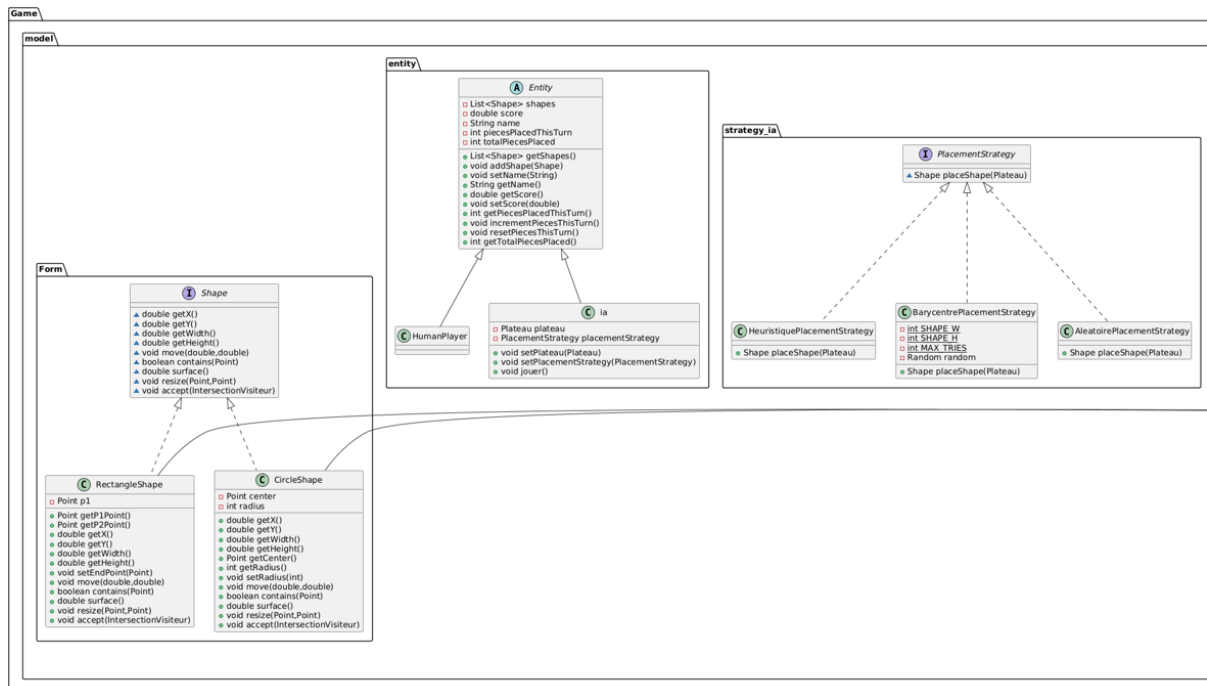


Figure 4: Diagramme UML GameModel Partie 1

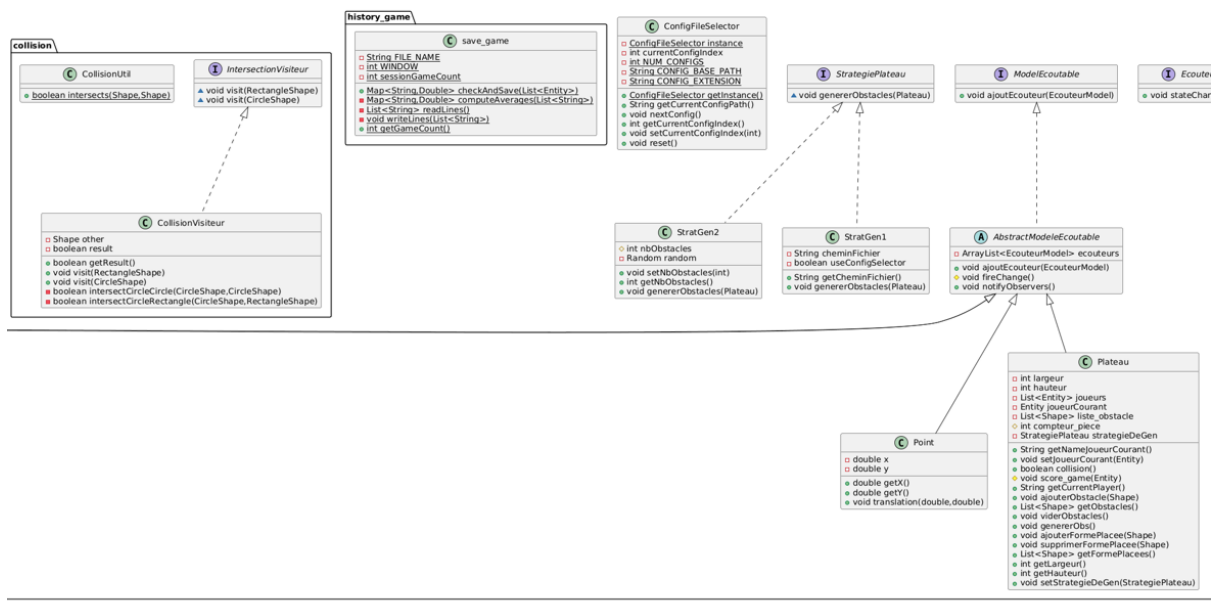


Figure 5: Diagramme UML GameModel Partie 2

3 Vue

3.1 GameMenu :



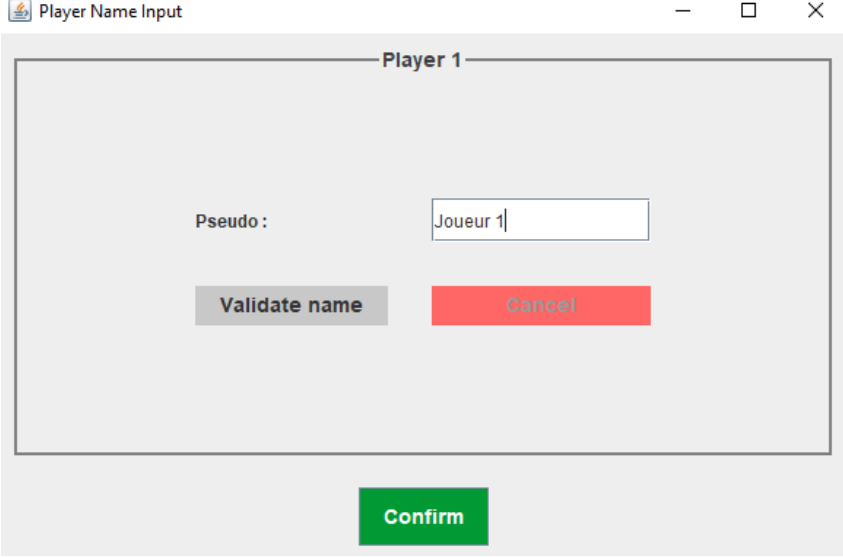
Figure 6: Visualisation du menu

Voici notre menu, il est constitué de 5 boutons. Tout d'abord nous avons un bouton "jouer" qui permet de lancer notre jeu. Ce bouton est équipé d'un écouteur d'événements. Pour que le jeu se lance il faut que les utilisateurs aient au préalable choisi leurs pseudos. Si ce n'est pas le cas une fenêtre s'affiche afin de prévenir les utilisateurs qu'il faut d'abord effectuer ce choix. Si les utilisateurs effectuent leurs choix et décident par la suite de changer les paramètres en l'occurrence le nombre de joueurs et qu'ils lancent le jeu, une nouvelle fenêtre apparaîtra pour leur dire "Veuillez sélectionner à nouveau les personnages après avoir modifié le nombre de joueurs".

En outre, nous avons un bouton "joueurs" qui permet aux utilisateurs de choisir leurs pseudos.

Nous avons également un bouton “paramètres” qui permet aux utilisateurs de définir les paramètres du jeu. Cette classe est également composée d’un bouton de sortie situé en bas à droite et d’un bouton pause situé en bas à gauche. Lorsque l’utilisateur appuie sur le bouton de sortie, une demande de confirmation apparaît et demande à l’utilisateur s’il veut vraiment quitter le jeu. Le bouton pause permet de stopper la lecture de notre musique ou de la reprendre.

3.2 GameCharacter :



The screenshot shows a window titled "Player Name Input" with standard window controls (minimize, maximize, close). Inside, there is a section labeled "Player 1". Below this label, the text "Pseudo :" is followed by a text input field containing "Joueur 1". Below the input field are two buttons: "Validate name" (grey) and "Cancel" (red). At the bottom of the window, centered, is a green "Confirm" button.

Figure 7: Visualisation de la sélection des pseudos



The screenshot shows the same "Player Name Input" window, but now it contains two side-by-side panels for "Player 1" and "Player 2". Each panel has a "Pseudo :" label, a text input field (containing "Joueur 1" and "Joueur 2" respectively), and "Validate name" (grey) and "Cancel" (red) buttons. The green "Confirm" button remains at the bottom center of the window.

Figure 8: Visualisation de la sélection des pseudos (2 joueurs)

Notre classe “**GameCharacter**” est une interface utilisateur où les joueurs peuvent entrer un pseudo.

3.3 GameSettings :

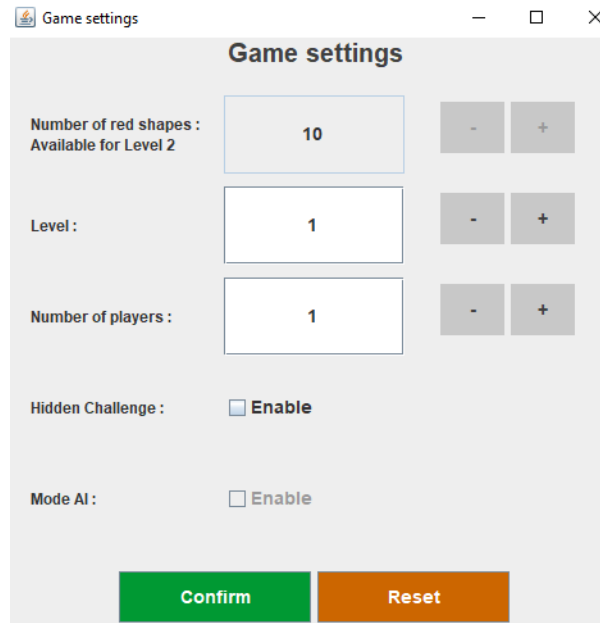


Figure 9: Visualisation des paramètres

Notre classe `GameSettings` permet à l'utilisateur de configurer plusieurs paramètres du jeu. Il peut notamment modifier le niveau, qui correspond aux différentes stratégies utilisées. L'utilisateur a également la possibilité d'activer le mode `hiddenChallenge`, qui consiste à cacher les formes après 10 secondes, ainsi que d'activer l'ia, uniquement dans le cas d'une partie à deux joueurs. Enfin, l'option des formes rouges est disponible uniquement lorsque le niveau est réglé sur 2.

3.4 GamePanel :

La classe `GamePanel` se charge de l'affichage graphique du plateau de jeu. En héritant de `JPanel` et en implémentant l'interface `EcouteurModel`, elle met à jour automatiquement l'affichage lorsque l'état du modèle change. Elle reçoit en paramètre une instance de `Plateau`, s'y enregistre comme écouteur, puis redessine son contenu à chaque modification grâce à la méthode `repaint()`.

Une de ses méthodes importantes est `startHiddenChallengeTimer()`. Cette méthode sert à afficher les obstacles pendant un court moment avant de les faire disparaître. Lorsqu'elle est appelée, les obstacles deviennent d'abord visibles et l'affichage du panneau est immédiatement mis à jour. Ensuite, un minuteur de 10 secondes est lancé. Une fois ce temps écoulé, les obstacles sont automatiquement masqués.

4 Contrôleur

4.1 Pattern Command

Nous avons implémenté le pattern **command** afin de gérer toutes les actions réalisés par l'utilisateur. L'interface `OperationCommand` définit les deux opérations principales : `operate()` pour exécuter une action et `compensate()` pour l'annuler.

La classe `CommandHandler` joue le rôle de gestionnaire des commandes et gère l'historique des actions à l'aide de deux listes (undo et redo). Lorsqu'une action est exécutée, elle est enregistrée dans l'historique. Cela permet donc de revenir en arrière ou d'annuler une action.

L'utilisateur dispose de différentes commandes telles que la création de rectangles et de cercles. Il peut également redimensionner les formes, les déplacer et les supprimer.

4.2 Pattern State

Nous avons mis en œuvre le pattern **State** afin de gérer les différents modes d'interaction de l'utilisateur avec le plateau. L'interface `StateController` définit les trois événements principaux liés à la souris : **`mousePressed`**, **`mouseDragged`** et **`mouseReleased`**. Chaque état correspond à une action spécifique du jeu (création de cercle, création de rectangle, déplacement, suppression ou redimensionnement d'une forme).

Ainsi les états de création permettent de construire progressivement une forme lors du déplacement de la souris, tandis que les états de modification (déplacement ou redimensionnement) mettent à jour dynamiquement les formes tout en vérifiant les collisions. Une fois l'action terminée, celle-ci est transformée en commande afin d'être enregistrée dans l'historique pour gérer les fonctionnalités d'undo/redo.

5 Engine

L'engine du jeu utilise le patron Chaîne de Responsabilité (Chain of Responsibility) pour gérer les événements déclenchés à chaque placement de pièce. La classe abstraite `GameHandler` définit la structure de chaque maillon : un attribut `nextHandler` et la méthode **`passToNext(session)`** qui transmet la session au maillon suivant s'il existe. Chaque sous-classe implémente sa propre logique dans **`handle(GameSession)`**.

La chaîne est construite dans **`GameSession.buildHandlerChain()`** comme ceci : `PlayerVerificationHandler` vers `GameTerminationHandler`. Après chaque placement d'une pièce par un joueur humain, **`GameSession.onPiecePlaced()`** déclenche **`handlerChain.handle(this)`**. Le `PlayerVerificationHandler` vérifie d'abord l'état du joueur courant (pièces restantes pour son tour, changement de joueur), puis passe la main au `GameTerminationHandler`. Ce dernier vérifie si le total de pièces placées a atteint la limite de la partie ($4 \times$ nombre de joueurs) ; si c'est le cas, il orchestre la fin de partie : sauvegarde des scores via **`save_game`**, affichage des moyennes sur 10 parties, et proposition de rejouer.

6 Répartition du travail

- implémentation de Game.command → Jérémy
- implémentation du Game.engine → Gabriel
- implémentation de Game.model → Youssef / Jérémy
- implémentation de Game.model.collision → Gabriel
- implémentation de Game.model.entity → Youssef
- implémentation de Game.model.form → Jérémy
- implémentation de Game.model.history_game → Gabriel
- implémentation de Game.model.stratConfig → Youssef
- implémentation de Game.model.strategy_ia → Gabriel
- implémentation de Game.model.stratGen → Youssef
- implémentation de Game.state → Jérémy
- implémentation de Game.view → Jérémy / Gabriel
- implémentation de Game.view.menu → Jérémy